



INSTITUTO POLITECNICO NACIONAL



**UNIDAD PROFESIONAL INTERDISCIPLINARIA EN
INGENIERÍA Y TECNOLOGÍAS AVANZADAS**

“Tarea 4: Propuesta de Fragmentación”

Peralta Ordoñez Jesús

Valencia Cedeño Marcos Gael

Vargas Clemente Leonel Zajid

Asignatura: Bases de Datos Distribuidas

Profesor: De la Cruz Sosa Carlos

Grupo: 3TM2

INDICE

Introducción	3
Grafo Relacional.....	4
Desarrollo	5
Propuesta 1	5
Propuesta 2	10
Conclusiones	13

Introducción

Para la tarea 3 de la materia Base de Datos Distribuidas se debe proponer un diseño de fragmentación para la Base de datos AdventuresWork2022, basado en uno de las 4 propuestas dadas por el profesor.

Para el desarrollo de esta Tarea fue escogida la siguiente propuesta:

/*

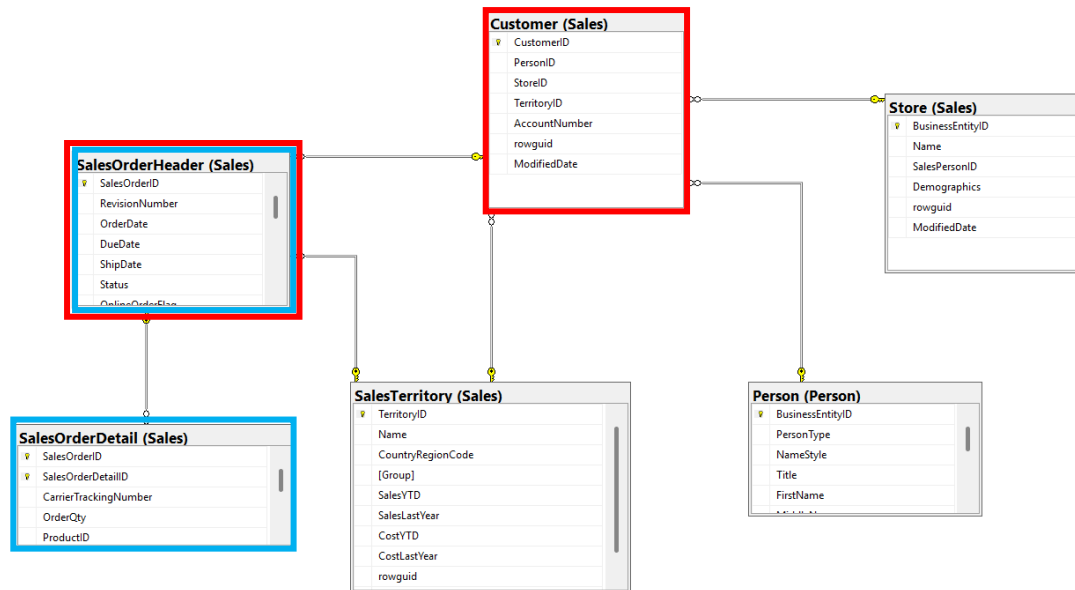
Ejercicio 3: Distribuir la cartera de clientes en los 5 nodos geográficos para que cada región administre y consulte de forma autónoma los datos de sus propios compradores. Tabla Sales.Customer, columna CustomerId (rangos).

- 1) Basado en la columna customid, en la que deben calcular los rangos para los predicados y en este sentido el conjunto M debe describir los rangos de cada región.
- 2) Basado en la columna territoryId que a su vez tiene que trabajarse con la tabla sales.Territory con la columna group

*/

Este ejercicio requiere del análisis de la tabla Sales.Customer dentro del modelo relacional en AdventureWorks2022 para el análisis de las relaciones entre tablas.

Grafo Relacional



Siendo Sales.Customer la tabla propietaria, observamos que se relaciona con 4 tablas, pero solo una de ellas tiene como ForeignKey la PrimaryKey de Sales.Customer la cual es Sales.SalesOrderHeader, esta tabla es una tabla miembro de Sales.Customer.

Esto es importante ya que al momento de empezar el proceso de fragmentación la relación entre tablas determina que tabla requieren de una fragmentación primaria y una fragmentación derivada. Ya sea fragmentación vertical u horizontal.

Una aclaración pertinente es que SalesOrderDetail es una tabla miembro de SalesOrderHeader, por lo que la fragmentación se propaga hasta SalesOrderDetail

Desarrollo

Propuesta 1

En la propuesta 1 usaremos el atributo CustomerID como rango para nuestra fragmentación, junto a la relación ya conocida de la tabla propietaria (Customer) y la tabla miembro (SalesOrderHeader).

$$\begin{aligned} \text{owner}(\text{FK_SalesOrderHeader_Customer_CustomerID}) &= \text{Sales.Customer} \\ \text{member}(\text{FK_SalesOrderHeader_Customer_CustomerID}) \\ &= \text{Sales.SalesOrderHeader} \end{aligned}$$

En la tabla Sales.Customer es necesario ver la cantidad de registros pertenecientes a esa tabla, también definir predicados simples para la fragmentación. Es importante recalcar que la unión de todos los conjuntos de predicados simples debe dar como resultado el conjunto original.

Consulta Total de Registros

```
use AdventureWorks2022
go
select Count(*) as TotalRegistros
from Sales.Customer
```

	TotalRegistros
1	19820

Consulta ID de Customers

```
use AdventureWorks2022
go
select Count(*) as TotalRegistros,
max(CustomerID) as PrimerID,
min(CustomerID) as UltimoID
from Sales.Customer
```

	TotalRegistros	PrimerID	UltimoID
1	19820	30118	1

Con estas consultas podemos ver que has más ID's que Registros, por lo que será necesario determinar un predicado simple que divida equitativamente los ID's en 5 nodos

Consulta numero de Customers en cada predicado

```
WITH Clientes AS
(
    SELECT
        CustomerID,
        ROW_NUMBER() OVER (ORDER BY CustomerID) AS NumFila
    FROM Sales.Customer
)
SELECT
    CASE
        WHEN NumFila BETWEEN 1 AND 3964 THEN 1
        WHEN NumFila BETWEEN 3965 AND 7928 THEN 2
        WHEN NumFila BETWEEN 7929 AND 11892 THEN 3
        WHEN NumFila BETWEEN 11893 AND 15856 THEN 4
        WHEN NumFila BETWEEN 15857 AND 19820 THEN 5
    END AS Nodo,
    MIN(CustomerID) AS CustomerID_Min,
    MAX(CustomerID) AS CustomerID_Max,
    COUNT(*) AS TotalClientes
FROM Clientes
GROUP BY
    CASE
        WHEN NumFila BETWEEN 1 AND 3964 THEN 1
        WHEN NumFila BETWEEN 3965 AND 7928 THEN 2
        WHEN NumFila BETWEEN 7929 AND 11892 THEN 3
        WHEN NumFila BETWEEN 11893 AND 15856 THEN 4
        WHEN NumFila BETWEEN 15857 AND 19820 THEN 5
    END
ORDER BY Nodo;
```

	Nodo	CustomerID_Min	CustomerID_Max	TotalClientes
1	1	1	14262	3964
2	2	14263	18226	3964
3	3	18227	22190	3964
4	4	22191	26154	3964
5	5	26155	30118	3964

Con esta consulta delimitamos los rangos utilizados para definir los predicados simples mediante una **CTE** (Clientes), en la cual recuperamos las tuplas de CustomerID. Con la función ROW_NUMBER() asignamos un valor secuencial a cada tupla según el orden de CustomerID, lo que permite trabajar con la cantidad real de registros aunque existan huecos entre los valores de CustomerID. Posteriormente, estos valores secuenciales permiten dividir las 19,820 tuplas en

grupos de 3,964 registros y obtener los valores mínimo y máximo de CustomerID que delimitan cada rango.

Conociendo estos valores es posible determinar los predicados simples necesario para la fragmentación.

$$\begin{aligned}
 P_r &= \{p_1, p_2, p_3, p_4, p_5\} \\
 p_1 &= CustomerId \leq 14262 \\
 p_2 &= CustomerId \leq 18226 \\
 p_3 &= CustomerId \leq 22190 \\
 p_4 &= CustomerId \leq 26154 \\
 p_5 &= CustomerId \leq 30118
 \end{aligned}$$

Una vez definidos estos términos es posible crear minitérminos con ellos de la siguiente manera

$$\begin{aligned}
 M &= \{m_1, m_2, m_3, m_4, m_5\} \\
 m_1 &= p_1 \wedge \neg(p_2) \wedge \neg(p_3) \wedge \neg(p_4) \wedge \neg(p_5) \\
 m_2 &= \neg(p_1) \wedge p_2 \wedge \neg(p_3) \wedge \neg(p_4) \wedge \neg(p_5) \\
 m_3 &= \neg(p_1) \wedge \neg(p_2) \wedge p_3 \wedge \neg(p_4) \wedge \neg(p_5) \\
 m_4 &= \neg(p_1) \wedge \neg(p_2) \wedge \neg(p_3) \wedge p_4 \wedge \neg(p_5) \\
 m_5 &= \neg(p_1) \wedge \neg(p_2) \wedge \neg(p_3) \wedge \neg(p_4) \wedge p_5
 \end{aligned}$$

Una vez que los minitérminos estén definidos será posible definir los fragmentos de nuestra tabla

$$\begin{aligned}
 Customer_1 &= \sigma_{m_1}(Customers) \\
 Customer_2 &= \sigma_{m_2}(Customers) \\
 Customer_3 &= \sigma_{m_3}(Customers) \\
 Customer_4 &= \sigma_{m_4}(Customers) \\
 Customer_5 &= \sigma_{m_5}(Customers)
 \end{aligned}$$

Lo que acabamos de realizar son todos los elementos necesarios para llevar a cabo una fragmentación horizontal primaria de Sales.Customer. Al ser Sales.Customer la relación propietaria sus fragmentos pueden utilizarse como criterios para fragmentar las tablas miembro de la relación

En este ejercicio Sales.SalesOrderHeader es una tabla miembro, ya que mantiene una relación con Sales.Customer a través de la Foreign_Key CustomerID, por lo que la fragmentación de Sales.Customer se propaga hacía Sales.SalesOrderHeader.

Para fragmentar una tabla miembro realizamos la operación de semi-join. En este caso un semi-join entre Sales.Customer y Sales.SalesOrderHeader con la condición de reunión de CustomerID

$$SalesOrderHeader_1 = SalesOrderHeader \bowtie Customer_1$$

$$SalesOrderHeader_2 = SalesOrderHeader \bowtie Customer_2$$

$$SalesOrderHeader_3 = SalesOrderHeader \bowtie Customer_3$$

$$SalesOrderHeader_4 = SalesOrderHeader \bowtie Customer_4$$

$$SalesOrderHeader_5 = SalesOrderHeader \bowtie Customer_5$$

El proceso de la fragmentación derivada se repite sobre SalesOrderDetail debido a la propagación de la fragmentación primaria sucedida en Sales.Customer. Para hacer esto debemos ejecutar semi-join's sobre los fragmentos generados de SalesOrderHeader.

$$SalesOrderDetail_1 = SalesOrderDetail \bowtie SalesOrderHeader_1$$

$$SalesOrderDetail_2 = SalesOrderDetail \bowtie SalesOrderHeader_2$$

$$SalesOrderDetail_3 = SalesOrderDetail \bowtie SalesOrderHeader_3$$

$$SalesOrderDetail_4 = SalesOrderDetail \bowtie SalesOrderHeader_4$$

$$SalesOrderDetail_5 = SalesOrderDetail \bowtie SalesOrderHeader_5$$

Para la propuesta 1 utilizamos el atributo CustomerID de la tabla Sales.Customer como atributo de fragmentación. A partir de este atributo determinamos los predicados simples y los minitérminos necesarios para realizar una fragmentación horizontal primaria de Sales.Customer.

Para determinar el conjunto mínimo y completo de predicados simples se utiliza el algoritmo COM_MIN. Este algoritmo analiza los predicados simples candidatos y determina cuáles son necesarios para distinguir los diferentes patrones de acceso a la relación. Si un predicado no aporta una nueva distinción relevante para los patrones de acceso, puede ser descartado por considerarse redundante. En nuestra propuesta, CustomerID es el único atributo utilizado como criterio de fragmentación, por lo que no fue necesario incorporar otros atributos para definir los fragmentos.

Propuesta 2

En la propuesta 2 usaremos el atributo TerritoryID como rango para nuestra fragmentación, junto a la relación ya conocida de la tabla propietaria (Customer) y la tabla miembro (SalesOrderHeader).

owner(FK_SalesOrderHeader_Customer_CustomerID) = Sales.Customer
member(FK_SalesOrderHeader_Customer_CustomerID)
= Sales.SalesOrderHeader

En la tabla Sales.Customer ejecutaremos consultas que nos proporcionen la información necesaria para el desarrollo de predicados simples y minitérminos de nuestra fragmentación.

```
SELECT
    Terr.TerritoryID,
    Terr.Name AS Territorio,
    Terr.CountryRegionCode,
    Terr.[Group]
FROM Sales.SalesTerritory AS Terr
LEFT JOIN Sales.Customer AS C
    ON C.TerritoryID = Terr.TerritoryID
GROUP BY
    Terr.TerritoryID,
    Terr.Name,
    Terr.CountryRegionCode,
    Terr.[Group]
ORDER BY
    Terr.TerritoryID;
```

	TerritoryID	Territorio	CountryRegionCode	Group
1	1	Northwest	US	North America
2	2	Northeast	US	North America
3	3	Central	US	North America
4	4	Southwest	US	North America
5	5	Southeast	US	North America
6	6	Canada	CA	North America
7	7	France	FR	Europe
8	8	Germany	DE	Europe
9	9	Australia	AU	Pacific
10	10	United Kingdom	GB	Europe

Con estos datos recuperados de la base de datos de AdventureWorks2022 podemos empezar a definir nuestro conjunto de predicados simples como:

$$\begin{aligned}
 P_r &= \{p_1, p_2, p_3, p_4, p_5, p_6, p_7, p_8, p_9, p_{10}\} \\
 p_1 &= TerritoryId = 1 \\
 p_2 &= TerritoryId = 2 \\
 p_3 &= TerritoryId = 3 \\
 p_4 &= TerritoryId = 4 \\
 p_5 &= TerritoryId = 5 \\
 p_6 &= TerritoryId = 6 \\
 p_7 &= TerritoryId = 7 \\
 p_8 &= TerritoryId = 8 \\
 p_9 &= TerritoryId = 9 \\
 p_{10} &= TerritoryId = 10
 \end{aligned}$$

Estos predicados simples sirven para generar el conjunto de minitérminos basados en la columna Group de Sales.SalesTerritory

$$\begin{aligned}
 m_1 &= p_1 \wedge p_2 \wedge p_3 \wedge p_4 \wedge p_5 \wedge p_6 \wedge \neg p_7 \wedge \neg p_8 \wedge \neg p_9 \wedge \neg p_{10} \\
 m_2 &= \neg p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge \neg p_4 \wedge \neg p_5 \wedge \neg p_6 \wedge p_7 \wedge p_8 \wedge \neg p_9 \wedge p_{10} \\
 m_3 &= \neg p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge \neg p_4 \wedge \neg p_5 \wedge \neg p_6 \wedge \neg p_7 \wedge \neg p_8 \wedge p_9 \wedge \neg p_{10}
 \end{aligned}$$

Con los minitérminos definidos podemos dar paso al desarrollo de nuestros fragmentos

$$\begin{aligned}
 Customer_1 &= \sigma_{m_1}(Customer) \\
 Customer_2 &= \sigma_{m_2}(Customer) \\
 Customer_3 &= \sigma_{m_3}(Customer)
 \end{aligned}$$

Donde $Customer_1$ sería la región de North America

Donde $Customer_2$ sería la región de Europe

Donde $Customer_3$ sería la región de Pacific

Para fragmentar una tabla miembro realizamos la operación de semi-join. En este caso un semi-join entre Sales.Customer y Sales.SalesOrderHeader con la condición de reunión de TerritoryID

$$\begin{aligned}
 SalesOrderHeader_1 &= SalesOrderHeader \ltimes Customer_1 \\
 SalesOrderHeader_2 &= SalesOrderHeader \ltimes Customer_2 \\
 SalesOrderHeader_3 &= SalesOrderHeader \ltimes Customer_3
 \end{aligned}$$

Como en la propuesta anterior, también es necesario propagar la fragmentación hasta SalesOrderDetail, generando los siguientes fragmentos de SalesOrderDetail

$$SalesOrderDetail_1 = SalesOrderDetail \ltimes SalesOrderHeader_1$$

$$SalesOrderDetail_2 = SalesOrderDetail \ltimes SalesOrderHeader_2$$

$$SalesOrderDetail_3 = SalesOrderDetail \ltimes SalesOrderHeader_3$$

Conclusiones

La fragmentación permite separar una relación (tabla) en diferentes subconjuntos, buscando distribuir la información de una manera más organizada y adecuada a las necesidades de cada nodo. Para que una fragmentación sea correcta debe cumplir con tres propiedades fundamentales: **completitud, exclusión y reconstrucción**, las cuales garantizan que no se pierda información, que los fragmentos no contengan datos innecesariamente repetidos y que sea posible obtener nuevamente la relación original.

El análisis del grafo relacional es fundamental para determinar la forma en que se encuentran relacionadas las tablas y distinguir entre relaciones propietarias y relaciones miembro. A partir de esta relación es posible determinar si se requiere una fragmentación **horizontal primaria o derivada**. En este trabajo se utilizó principalmente la fragmentación horizontal, la cual divide una relación en subconjuntos de **tuplas mediante predicados**, estableciendo las condiciones que determinan a qué fragmento pertenece cada registro. Estos predicados permiten construir minitérminos que posteriormente sirven para definir los diferentes fragmentos.

Finalmente, la fragmentación no solamente permite dividir una tabla de manera independiente, sino también conservar las relaciones existentes entre las diferentes tablas. Cuando una relación propietaria se fragmenta, sus fragmentos pueden utilizarse como criterio para fragmentar las relaciones miembro mediante el operador **semi-join**. De esta manera, la distribución de los datos mantiene la relación lógica entre las tablas y permite que cada fragmento contenga la información correspondiente, facilitando su administración y consulta de acuerdo con las necesidades de cada nodo.